Kairos: A 32-Core Programmable SIMD GPU with HDMI Video Pipeline

HW–SW Co-design for Fixed-Point GPGPU Compute on Xilinx Zynq-7000 SoC

Shawaiz Haider Muhammad Hassan Bilal Arslan Nazar

SEECS, National University of Sciences and Technology

{shaider, mbilal, anazar}.bee23seecs@seecs.edu.pk

Abstract

This report describes the architecture and implementation of a 32-core, single-instruction multiple-data (SIMD) general-purpose GPU (GPGPU) realised on the Programmable Logic (PL) fabric of a Xilinx Zynq-7000 SoC (ZC706 evaluation board). The compute engine consists of 32 multiply-accumulate (MAC) units operating in Q8.24 fixed-point arithmetic at 74.25 MHz, yielding a peak throughput of 2.37 GOPS. A custom 32-bit ISA with 16 opcodes drives the datapath through a dedicated controller, while a 3072-bit internal data bus routes operands to all MAC cores simultaneously. The design leverages the Zynq's heterogeneous architecture: the ARM Cortex-A9 Processing System (PS) handles FAT-32 SD card I/O, DDR3 DMA transfers for 1280×720 video and 48 kHz PCM audio, and system control; the PL implements the full GPU pipeline including frame buffering and HDMI signal generation via an ADV7511 transmitter. Two representative GPU programs—a zoomable Mandelbrot set renderer and a 3×3 fixed-point matrix multiplier—are presented, along with a performance analysis and discussion of design challenges.

1. System Overview

The system is a hardware–software co-design partitioned across the two domains of the Zynq-7000 SoC. The PL (Programmable Logic, FPGA fabric) is clocked at 74.25 MHz, derived from the board's 200 MHz differential clock via an MMCME2 PLL configured with a VCO frequency of 200 MHz × (37.125/8) = 928.125 MHz and a CLKOUT0 divider of 12.5, matching the 74.25 MHz pixel clock mandated by the 720p60 HDMI standard. The ARM Cortex-A9 PS runs at 100 MHz and communicates with PL peripherals exclusively through AXI interconnects.

At the highest level the system comprises:

- A 32-MAC SIMD compute engine with a custom ISA controller;
- A 1.84 Mb dual-port VRAM frame buffer with hardware double-buffering;
- A 720p HDMI output pipeline (video timing controller + ADV7511 I2C initialiser);
- An AXI Video DMA (VDMA) path feeding pre-rendered frames from PS DDR3 to the display;
- An AXI DMA path streaming 48 kHz stereo PCM audio to the ADV7511 SPDIF input;
- PS-side bare-metal firmware for SD card media loading and AV synchronisation.

2. Hardware Design (Programmable Logic)

2.1 Datapath

The compute datapath consists of 32 MAC units instantiated in parallel via a `generate` loop. Each MAC unit accepts three 32-bit Q8.24 fixed-point operands (A, B, C) and computes:

$$\text{out} = \left\lfloor \frac{A \times B}{2^{24}} \right\rfloor + C$$

where the full-precision 64-bit intermediate product is right-shifted by `FIXED_POINT` = 24 bits before addition. The operand bus connecting the controller to all MACs in parallel is $32 \times 3 \times 32 = 3072$ bits wide—the defining structural constant of the entire design.

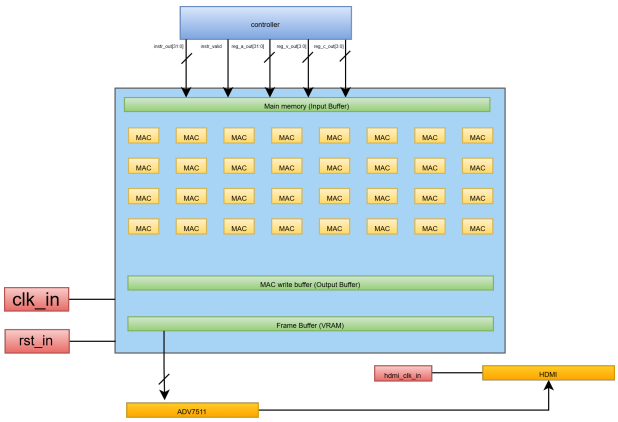

Figure 1: Datapath

Each MAC instance exposes three control signals: `valid_in` (load new A, B), `c_valid_in` (replace or accumulate C), and `output_can_be_valid_in` (gate whether the current result should propagate downstream). This allows the controller to chain multiply-accumulate operations across multiple instruction cycles, building up dot products without architectural stalls.

The 32 MAC outputs are routed into a **MAC Write Buffer** (`fma_write_buffer`), which collects three consecutive valid output words per MAC over three clock cycles. Because each MAC produces a 32-bit result, and three such results per MAC are needed to represent the three output "phrases" of a computation, the write buffer assembles a complete $32 \times 3 \times 32 = 3072$-bit line before asserting `line_valid` to the memory subsystem.

2.2 Controller

The controller is ISA-agnostic with respect to the compute datapath: it simply fetches 32-bit instructions from a true dual-port BRAM (100 entries deep), decodes them, drives the memory module, and manages a sixteen-entry, 32-bit general-purpose register file. The register file stores loop counters, stride coefficients, and coordinate origins required by GPU programs.

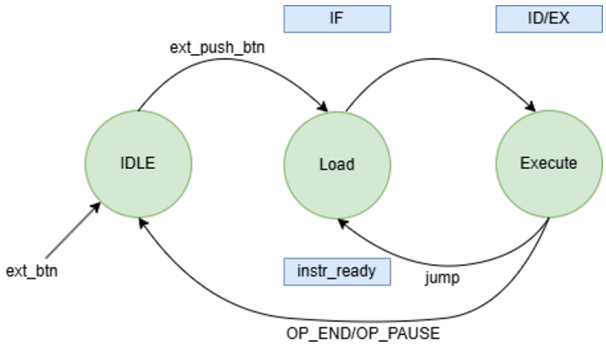

Figure 2: Finite State Machine for Controller

Instruction memory is implemented as a dual-port BRAM to allow simultaneous fetch and prefetch in adjacent addresses, reducing effective fetch latency. The controller generates a one-cycle-wide `instr_valid` pulse for each issued instruction, which is consumed by the memory module on the same cycle.

SIMD branch divergence is handled architecturally through the `OR` instruction (opcode `1101`), which operates as a per-lane predication mask. Because all 32 MACs execute the same instruction sequence in lockstep, divergent conditions (e.g. whether a Mandelbrot iterate has escaped) cannot cause individual MACs to branch. Instead, each MAC maintains an iteration counter stored in the memory module, and the `OR` instruction performs a conditional write to each lane's counter based on that lane's convergence state. This predictive masking avoids lane deactivation and keeps all 32 MACs active at all times.

2.3 Memory Architecture

The memory subsystem (`memory.sv`) serves as the interface between the controller, the MAC array, and the write buffer. Its internal structure is:

Main Memory (L1 Scratchpad) A 3072-bit register file synthesised from D flip-flops rather than BRAM. This choice is deliberate: BRAM has a minimum read latency of one or two cycles, which would stall the MAC pipeline. A flat register bank allows instantaneous read and write in a single clock edge, at the cost of LUT/FF resources. The scratchpad holds one full 3072-bit operand line: three 32-bit values (A, B, C) for each of the 32 MACs. Individual 32-bit slots are addressed using parameterised macro indexing (`BRAM_TEMP(fma_id, abc)`).

Write Buffer Register A second 3072-bit register that latches the assembled output of the write buffer on each `line_valid` pulse. From this register, the `LOADB` instruction can shuffle individual per-MAC output phrases back into the scratchpad as inputs for the next computation.

VRAM (Frame Buffer). A dual-port BRAM sized to hold the current iteration counts for all 1280×720 pixels. The GPU writes to Port A addressed by `mandelbrot_addr_out`, while the HDMI display controller reads from Port B addressed by the running pixel counter (`hcount`, `vcount`). Hardware double-buffering is achieved by toggling the MSB of the BRAM address on a `FBSWAP` instruction, which atomically swaps the active display region without tearing.

Addressing. The L1 scratchpad slots are indexed as:

$$\text{slot}(i,k) = \text{LINE\_WIDTH} - (i \cdot 3 + k + 1) \cdot \text{WORD\_WIDTH}$$

for MAC index $i \in [0, 31]$ and operand slot $k \in \{0, 1, 2\}$ representing A, B, C respectively.

2.4 Video Signal Generation and ADV7511

2.4.1 Video Timing Controller

The `video_sig_gen` module generates 720p60 (1280×720 at 74.25 MHz) timing signals as per CEA-861 specifications. The horizontal total is:

$$H_{\text{total}} = 1280 + 110 + 40 + 220 = 1650 \text{ pixels}$$

and the vertical total is:

$$V_{\text{total}} = 720 + 5 + 5 + 20 = 750 \text{ lines}$$

yielding a frame rate of $74.25\,\text{MHz}/(1650 \times 750) = 60.0\,\text{Hz}$. The module counts pixel positions using a pair of counters (`hcount`, `vcount`) and derives `hs_out`, `vs_out`, `ad_out` (active draw) and `nf_out` (new frame pulse) combinatorially. A six-bit `fc_out` provides a modulo-60 frame counter.

2.4.2 ADV7511 Initialisation Sequencer

The ADV7511 HDMI transmitter on the ZC706 is accessed through a PCA9548 I2C multiplexer. The `adv7511_init` module implements a bitbanged I2C master state machine at 100 kHz (CLK_DIV = 74250000/(100000 × 2) = 371 cycles) running entirely in PL logic, independent of the PS I2C controllers.

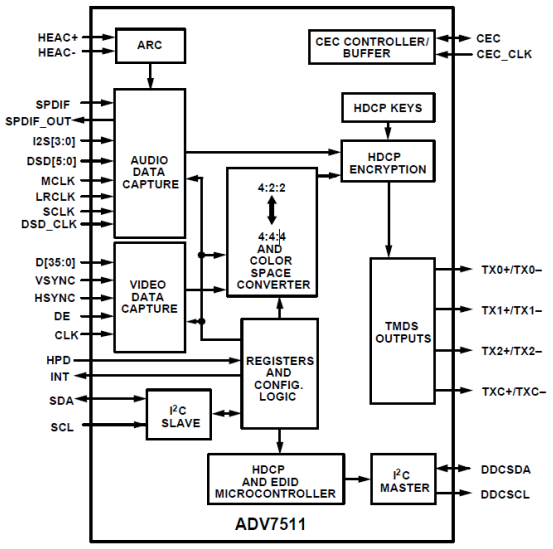

Figure 3: ADV-7511 Architecture

After a mandatory 200 ms startup delay (14 850 000 cycles), the sequencer issues 19 transactions. The key register writes are summarised in Table 1, with the full sequence including:

1. PCA9548 channel 1 select (address 0x74, data 0x02);
2. Hot-plug detect (HPD) poll at register 0x42[6]—the sequencer loops with a 742 500-cycle backoff until HPD is asserted;
3. Power-up by clearing bit 6 of register 0x41 (0x10);
4. Seven fixed-register writes (0x98–0xF9) per ADV7511 programming guide;
5. General Control packet enable (0x40 ← 0x80);
6. Input format: register 0x15 = 0x00 (24-bit RGB 4:4:4), register 0x16 = 0x34 (Input Style 1), register 0x17 = 0x02 (16:9 aspect ratio);
7. AVI InfoFrame: 0x55 = 0x00 (RGB output), 0x56 = 0x28 (16:9 AVI);
8. Clock delay 0xBA = 0x60;
9. HDMI mode: 0xAF = 0x02 (HDMI, not DVI).

NACK handling is built into the state machine: any NACK on the ACK phase triggers a STOP condition followed by a full retry from transaction 0, ensuring robust initialisation in the presence of marginal bus conditions.

Table 1: Key ADV7511 Register Writes

Reg	Value	Field	Purpose
0x41	0x10	[6]=0	Power up
0x15	0x00	[3:0]	24-bit RGB 4:4:4 input
0x16	0x34	[5:4]	Input Style 1 (R/G/B)
0x17	0x02	[1]	16:9 aspect ratio
0x40	0x80	[7]	Enable GC packet
0x55	0x00	—	AVI InfoFrame: RGB
0x56	0x28	—	AVI InfoFrame: 16:9
0xAF	0x02	[1]	HDMI mode

2.5 Instruction Set Architecture

The custom ISA is 32 bits wide, big-endian, with a fixed 4-bit opcode field in bits [0:3]. The instruction word is partitioned as:

```
[0:3] opcode | [4:7] reg_a | [8:23] imm16 |
       [24:27] reg_b | [28:31] reg_c
```

Table 2 provides the complete opcode listing.

Two ISA features merit particular attention. First, the `ADDI` and `LOADI` instructions support 32-bit immediates through assembler-level expansion into two back-to-back 16-bit instructions. For `ADDI`, when the lower 16 bits have their sign bit set, the assembler increments the upper half by one to cancel the sign-extension artefact—ensuring exact 32-bit two's-complement semantics across the two-instruction sequence. Second, the `LOAD` instruction implicitly fans out an affine initialisation across all 32 MAC lanes in a single cycle: MAC lane i receives $B + i \cdot C$, eliminating a 32-iteration setup loop for coordinate initialisation.

Table 2: Complete ISA Opcode Table

Opcode	Mnemonic	Operands	Description
0000	NOP	—	No operation. One-cycle pipeline bubble.
0001	END	—	Halt program execution.
0010	XOR	rA, rB	rA ← rA ⊕ rB. Idiomatic zero (xor r0 r0).
0011	ADDI	rA, rB, imm16	rA ← rB + sign_ext(imm16). 32-bit immediates expand to two instructions; the assembler compensates for sign-extension carry automatically.
0100	BGE	rA, rB	Branch if rA ≥ rB.
0101	JUMP	imm16	Unconditional jump to BRAM address imm16. Assembler resolves symbolic labels to expanded BRAM addresses to account for multi-word instructions.
0110	FBSWAP	—	Toggle frame buffer MSB. Atomically swaps the active HDMI display region (hardware double buffering).
0111	LOADI	rA, imm16	Load sign-extended 16-bit immediate into L1 scratchpad slot rA. If reg_c[0]=1 (LOAD HIGH flag), writes the immediate into bits [31:16] of the target slot only, enabling 32-bit immediate construction in two instructions.
1000	ADD	rA, rB, rC	rA ← rB+rC (controller register file only). Used for stride accumulation.
1001	LOADB	s1, s2, s3	Shuffle MAC write buffer outputs into L1 scratchpad slots A, B, C per MAC. Shuffle codes: 0=zero, 1–3=phrase[0–2], 4–6=2×phrase[0–2], 13–15=negated phrase[2–0]. Enables in-place operand routing without explicit moves.
1010	LOAD	slot, rB, rC	For each MAC i: bram_temp[i][slot] ← rB $+i\times$ rC. Broadcasts an affine coordinate sweep across all 32 lanes in one instruction.
1011	WRITEB	c_flag, v_flag	Commit L1 scratchpad to L1 BRAM at address imm16. c_flag controls C replacement; v_flag gates valid_out to the write buffer.
1100	WRITE	c_flag, v_flag	Transfer L1 scratchpad directly to MAC inputs. Issues abc_valid_out to trigger computation. c_flag=1 replaces C; c_flag=0 accumulates into the running C register.
1101	OR	iter_val	SIMD divergence mask. For each MAC lane i: if the lane's iteration counter is at maximum *and* its write buffer output (phrase 2, magnitude) exceeds 4.0 in Q8.24, update the counter to controller_reg_a[5:2]. This is the escape-condition test for Mandelbrot.
1110	SENDITERS	rX, rY	Commit current per-lane iteration counts to the frame buffer at address HEIGHT × rX + rY; reset internal counters to −1.
1111	PAUSE	—	Suspend execution until the PS asserts continue_in. Used for inter-frame synchronisation.

3. Bridging Hardware and Software (PS–PL Interface)

3.1 System Block Diagram

The Zynq PS operates at 100 MHz and exposes AXI master and slave ports to the PL fabric through the PS7 hard IP block. The PL-side logic runs at 74.25 MHz. Clock-domain crossing is managed by the AXI interconnect IP, which inserts handshaking registers where necessary. A Processing System Reset IP generates synchronised resets from the PS reset controller to all PL peripherals.

3.2 AXI Video DMA (VDMA)

A Xilinx AXI VDMA instance (MM2S direction only) reads frame data from PS DDR3 memory and streams it to the PL video pipeline. The VDMA is configured with a single frame buffer at physical address 0x01000000, stride equal to FRAME_WIDTH × PIXEL_BYTES = 3840 bytes, and circular buffer mode enabled. Once started, the VDMA continuously rereads the same frame address, so the PS firmware need only copy a new frame into DDR3 and flush the D-cache before the VDMA's next frame read.

3.3 AXI DMA (Audio)

A simple AXI DMA instance (MM2S, simple transfer mode) feeds 48 kHz stereo PCM audio data from

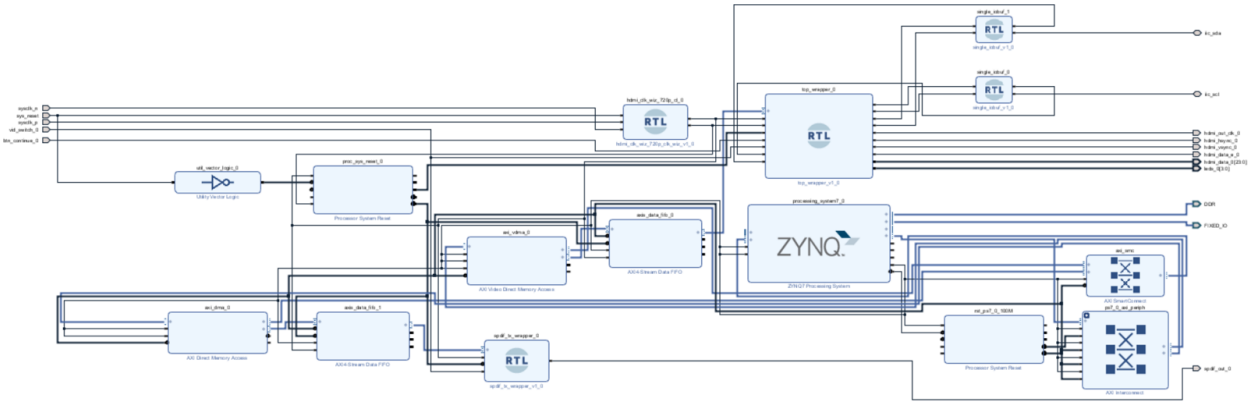

Figure 4: Top Block Diagram

DDR3 (base address `0x03000000`) to the ADV7511's SPDIF input. Each transfer moves 3840 bytes, corresponding to exactly 20 ms of audio at 48 kHz × 2 channels × 2 bytes/sample. The DMA is polled in software to determine when the FIFO has drained, at which point the next chunk is queued.

3.4 AXI FIFOs and Interconnect

AXI Stream FIFOs buffer data between the DMA engines and PL-side consumers (HDMI and SPDIF), absorbing burst mismatches between the DDR3 bandwidth and the steady-state consumption rates. The video path requires continuous 74.25 MHz throughput at $3 \times 8 = 24$ bits/cycle = 178.2 MB/s, well within the VDMA's 1 GB/s DDR3 bandwidth budget.

4. Software Design (Processing System)

The PS runs bare-metal firmware (no OS) compiled with Xilinx SDK/Vitis against the BSP for the ZC706. The program flow is as follows:

1. SD Card Initialisation

The FatFs library mounts the SD card's FAT-32 partition at `"0:/"` using the Xilinx SD controller driver. Failure to mount is a non-recoverable error.

2. Audio Load

`AUDIO.BIN` is read sequentially into DDR3 at `0x03000000` using `f_read`. The file is raw 48 kHz, 16-bit stereo PCM—no header, no codec. The full file is loaded to avoid SD latency during playback.

3. Video Load

`VIDEO.BIN` is read in 5-frame chunks (each $5 \times 1280 \times 720 \times 3 = 13.824$ MB) into DDR3 at `0x04000000`. During each chunk read, a byte-swap loop converts BGR → RGB in-place (the video was encoded as BGR). Each chunk is D-cache flushed immediately after swap so the VDMA, which bypasses the cache, observes coherent data.

4. Hardware Initialisation

After all media is resident in DDR3, the AXI VDMA and AXI Audio DMA are initialised. The VDMA is started in circular mode pointing at `DISPLAY_BUF_A`. An initial 3840-byte DMA transfer pre-fills the audio FIFO.

5. AV Synchronisation Loop

The main loop implements audio-master synchronisation:

1. The AXI DMA is polled. When idle, a new 3840-byte audio chunk is queued. Each chunk is 20 ms of audio, so two chunks correspond to exactly one video frame at 25 FPS.
2. The expected video frame index is derived from the audio chunk counter: $f_{\text{expected}} = \lfloor n_{\text{audio}}/2 \rfloor$.
3. A 30 ms inter-frame accumulator gate prevents back-to-back heavy DDR3 memcpy calls (each `present_frame` copies 2.76 MB), which would stall the AXI interconnect and cause audio underruns.
4. When the gate permits and $f_{\text{video}} < f_{\text{expected}}$, `present_frame()` copies the next frame from storage to `DISPLAY_BUF_A` and flushes the D-cache. The VDMA's next frame read then picks

up the new data.
5. Track looping is handled by resetting both chunk and frame indices when the audio pointer would overrun `audio_file_size`.

Diagnostic counters (successful DMA transfers, failures, busy skips, video frame index, DMA status register) are printed over UART every 2000 loop iterations ($\approx 2\,\text{s}$).

5. Programming the GPU

5.1 Mandelbrot Set Renderer

The Mandelbrot set is defined as the set of points $c \in \mathbb{C}$ for which the orbit of $z_{n+1} = z_n^2 + c$, with $z_0 = 0$, remains bounded. Each of the 32 MAC lanes processes one complex coordinate in parallel; 32 adjacent y-coordinates sharing the same column x are dispatched per outer loop iteration. The escape test $|z|^2 > 4$ is evaluated by the `OR` instruction using the real and imaginary outputs accumulated in the write buffer.

Coordinates are represented in Q8.24 fixed-point. The viewport maps to $x_{\min} = -2.72$, $y_{\min} = -1.25$, $\Delta x = \Delta y \approx 3.47 \times 10^{-3}$. After each rendered frame, the viewport is nudged inward and the step size decremented to produce a continuous zoom sequence.

All 32 lanes execute every iteration unconditionally—escaped lanes retain their recorded count and continue computing, keeping the MAC array fully utilised. This trades wasted compute on escaped lanes for a branchless, constant-latency inner loop. Detailed implementation has been given in Algorithm 1 at page 7.

5.2 3×3 Fixed-Point Matrix Multiplication

Matrix multiplication $R = M_1 M_2$ requires $R_{ij} = \sum_{k=0}^{2} M_1[i,k] \cdot M_2[k,j]$. The GPU assigns one MAC lane per output element in a row: MAC 0 computes R_{i0}, MAC 1 computes R_{i1}, MAC 2 computes R_{i2}. Each of the three k-terms is a separate `WRITE` instruction; the `c_flag` field selects whether the MAC replaces or accumulates its running C register, implementing the dot-product loop without an explicit counter.

Repeating this block for rows $i \in \{0, 1, 2\}$ yields the complete result matrix R at BRAM addresses 100–102. The write buffer flush padding is an artefact of the three-phrase collection window; the dummy writes use zeroed A inputs and a fresh C so they do not corrupt any accumulated result. Detailed implementation has been given in Algorithm 2 at page 8.

6. Performance Analysis

6.1 Peak Throughput

Each MAC unit executes one multiply-accumulate per clock cycle. With 32 active MACs at 74.25 MHz:

$$\text{Throughput} = 32 \times 74.25 \times 10^6 = 2.376 \times 10^9 \approx 2.37\,\text{GOPS}$$

This figure is the raw silicon compute rate; effective application throughput is lower due to instruction overhead, memory latency, and the pipeline drain imposed by the write buffer's three-cycle collection window.

6.2 Mandelbrot Render Time

At 1280×720 resolution with a maximum of 63 iterations and 32 MACs processing 32 y-coordinates in parallel:

- Per-iteration inner loop: approximately 12 instructions (6 `WRITE`/`LOADB` pairs plus 2 `OR`/`ADDI`);
- Outer loop iterations per frame: $1280 \times \lceil 720/32 \rceil = 1280 \times 23 = 29\,440$;
- Worst-case instruction cycles per frame (all pixels at max iters): $29\,440 \times 63 \times 12 \approx 22.2\,\text{M}$ cycles;
- At 74.25 MHz: $\approx 0.30\,\text{s}$ worst-case render time.

In practice, measured render rate is approximately **0.7 FPS** for typical Mandelbrot views, accounting for the mix of escaped and boundary pixels, loop overhead, and write buffer flush cycles.

6.3 Memory Bandwidth

The 3072-bit internal scratchpad bus delivers:

$$74.25\,\text{MHz} \times 3072\,\text{bits} \approx 228\,\text{GB/s}$$

of internal operand bandwidth per clock, which entirely eliminates memory bandwidth as a bottleneck for the compute engine. VRAM write bandwidth is bounded by the MAC write buffer output rate: one 3072-bit line per three cycles, or $74.25/3 \approx 24.75\,\text{M}$ line-writes per second.

Algorithm 1: Mandelbrot Set Renderer (32-lane SIMD)

Input: $x_{\min}$, $y_{\min}$, Δx, Δy, $N_{\max}$ (max iterations)
Output: Per-pixel escape counts written to VRAM

for *each column* $x_{idx} \leftarrow 0$ **to** $W-1$ **do**
  $c_x \leftarrow x_{\min} + x_{\text{idx}} \cdot \Delta x$
  for *each row-block* $y_{idx} \leftarrow 0$ **to** $H-1$ ***step*** 32 **do**
    ▷ `Broadcast: lane` i `gets` $c_{y,i} = y_{\min} + (y_{\text{idx}} + i) \cdot \Delta y$
    $\mathbf{c_y}[0..31] \leftarrow$ `LOAD` $y_{\min}$, Δy
    $\mathbf{z_x}[0..31] \leftarrow \mathbf{0}$; $\mathbf{z_y}[0..31] \leftarrow \mathbf{0}$
    $\mathbf{iters}[0..31] \leftarrow N_{\max}$ ▷ `All lanes active`
    for $n \leftarrow 0$ **to** $N_{max}-1$ **do**
      ▷ $z'_x = z_x^2 - z_y^2 + c_x$ `via MAC chain`
      $\mathbf{t_1} \leftarrow \mathbf{z_x} \times \mathbf{z_x} + c_x$
      $\mathbf{t_2} \leftarrow \mathbf{z_y} \times \mathbf{z_y}$
      $\mathbf{z'_x} \leftarrow \mathbf{t_1} - \mathbf{t_2}$
      ▷ $z'_y = 2 z_x z_y + c_y$ `via MAC chain`
      $\mathbf{z'_y} \leftarrow 2\,(\mathbf{z_x} \times \mathbf{z_y}) + \mathbf{c_y}$
      $\mathbf{z_x} \leftarrow \mathbf{z'_x}$; $\mathbf{z_y} \leftarrow \mathbf{z'_y}$
      ▷ `OR: per-lane escape test (no branching)`
      for *lane* $i \leftarrow 0$ **to** 31 **do**
        if $\mathbf{iters}[i] = N_{max}$ ***and*** $\mathbf{z_x}[i]^2 + \mathbf{z_y}[i]^2 \geq 4$ **then**
          $\mathbf{iters}[i] \leftarrow n$ ▷ `Record escape iteration`
    `SENDITERS`($\mathbf{iters}$, x_{idx}, y_{idx}) ▷ `Commit to VRAM`
`FBSWAP` ▷ `Atomically swap display buffer`
`PAUSE` ▷ `Yield to PS; apply zoom parameters`
$x_{\min} \mathrel{+}= \delta_x$;
$y_{\min} \mathrel{+}= \delta_y$;
$\Delta x \mathrel{-}= \epsilon$;
$\Delta y \mathrel{-}= \epsilon$
goto Frame Loop

6.4 Area and Resource Utilisation

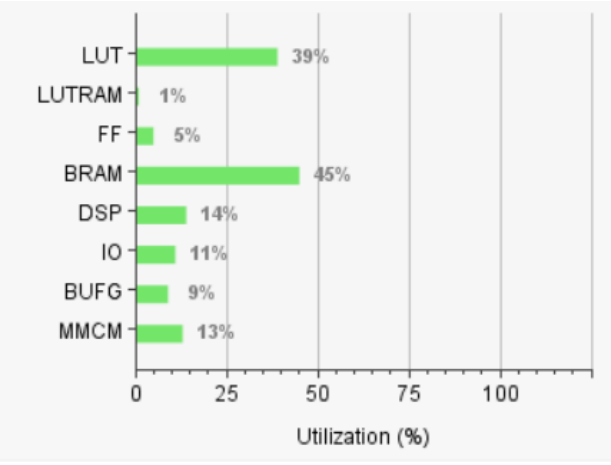

Figure 5: Resource Utilization on Xilinx ZC706

The design targets the XC7Z045 device on the ZC706. Dominant resource consumers are:

- **32 MAC units**: each instantiates one DSP48E1 for the 32×32 multiply. Total: 32 DSPs;
- **3072-bit scratchpad**: ≈ 3072 flip-flops, no BRAM;
- **Frame buffer VRAM**: 1.84 Mb dual-port BRAM (≈ 29 RAMB36E1 blocks at 64-bit width);
- **Instruction BRAM**: one RAMB36E1 (100 × 32-bit, true dual-port).

6.5 Power Consumption

On-chip power was estimated by Xilinx Vivado using post-implementation activity data. Total device power is 3.124 W, partitioned into 2.880 W dynamic (92%) and 0.244 W static (8%). Table 3 breaks down the dynamic component by resource class.

Algorithm 2: 3×3 Fixed-Point Matrix Multiply (Row i)

Input: $M_1[i, 0..2]$,
$M_2[0..2,\ 0..2]$ loaded into L1 scratchpad slots
Output: Row i of R committed to L1 BRAM

▷ --- Term $k = 0$: initialise accumulators ---
1 $\text{MAC}_j.A \leftarrow M_1[i, 0]$; $\text{MAC}_j.B \leftarrow M_2[0, j]$ $\forall j \in \{0, 1, 2\}$
2 $\text{MAC}_j.C \leftarrow 0$
3 WRITE(c_flag=1, v_flag=0) ▷ Replace C; result internal

▷ --- Term $k = 1$: accumulate ---
4 $\text{MAC}_j.A \leftarrow M_1[i, 1]$; $\text{MAC}_j.B \leftarrow M_2[1, j]$ $\forall j \in \{0, 1, 2\}$
5 WRITE(c_flag=0, v_flag=0) ▷ Accumulate into C

▷ --- Term $k = 2$: accumulate and push to write buffer ---
6 $\text{MAC}_j.A \leftarrow M_1[i, 2]$; $\text{MAC}_j.B \leftarrow M_2[2, j]$ $\forall j \in \{0, 1, 2\}$
7 WRITE(c_flag=0, v_flag=1) ▷ Accumulate; assert valid_out

▷ --- Flush write buffer (requires 3 valid cycles total) ---
8 $\text{MAC}_j.A \leftarrow 0$
9 WRITE(c_flag=1, v_flag=1) ▷ Dummy cycle 2
10 WRITE(c_flag=1, v_flag=1) ▷ Dummy cycle 3; buffer flushes
11 WRITEB(addr=$100 + i$) ▷ Commit 3072-bit line to L1 BRAM

Table 3: On-Chip Dynamic Power Breakdown

Resource	Power (W)	Share (%)
PS7 (ARM + DDR3)	1.572	55
Signals	0.660	23
Logic (LUTs/FFs)	0.327	11
MMCM (PLL)	0.116	4
Clocks	0.090	3
DSP48 (MAC units)	0.064	2
BRAM	0.030	1
I/O	0.020	1
Total Dynamic	**2.880**	**100**

The PS7 block dominates at 55% of dynamic power, driven primarily by DDR3 controller activity during SD card loading and AXI DMA video streaming. On the PL side, the signal routing network (23%) exceeds logic power (11%), which is expected given the 3072-bit wide internal operand bus connecting the MAC array—long nets at high toggle rates are the primary switching load. The 32 DSP48E1 MAC units contribute only 0.064 W (2%), confirming that computation is not the dominant power sink; data movement is. BRAM (1%) and I/O (1%) are negligible. The MMCM accounts for 4%, reflecting the single clock domain running at 74.25 MHz.

6.6 Timing Closure

Post-implementation timing was analysed using Vivado's static timing analysis engine across 70 327 timing endpoints. Results are summarised in Table 4.

Table 4: Post-Implementation Setup Timing Summary

Metric	Value
Worst Negative Slack (WNS)	+0.27 ns
Total Negative Slack (TNS)	0 ns
Failing Endpoints	0
Total Endpoints Analysed	70 327

The design meets timing closure at 74.25 MHz with a WNS of +0.27 ns—a margin of approximately 2% of the 13.47 ns clock period. TNS is exactly zero, indicating no path violates the constraint. The tight margin is attributable to the 3072-bit scratchpad bus: driving a wide combinational fan-out from a single always_ff block across 32 MAC instances creates long routing paths that consume most of the timing budget. Future iterations targeting higher clock rates would benefit from pipelining the scratchpad output stage or partitioning the bus into two independently registered halves.

7. Challenges

32-Bit Immediate Encoding. The ISA's 16-bit immediate field cannot encode the Q8.24 fixed-point constants required by the Mandelbrot program (e.g. $x_{\min} = -45\,634\,027$). Extending the instruction width was rejected to maintain 32-bit alignment of the instruction BRAM. The solution—two-instruction expansion with sign-extension compensation in the assembler—requires careful tracking of expanded

BRAM addresses for jump target resolution. A two-pass assembler was implemented: pass 1 maps source line indices to expanded BRAM addresses; pass 2 emits machine code and patches jump targets.

SIMD Divergence. The lock-step SIMD model is incompatible with per-lane early exit. A conventional approach would deactivate lanes that have already escaped, but this complicates the control logic and reduces throughput for non-divergent workloads. The `OR` instruction was designed as a specialised predication mechanism that writes the escape iteration count per lane while keeping all MACs active. The tradeoff is that escaped lanes continue performing (wasted) computation, but the instruction count per iteration remains constant and the controller stays simple.

LOADI vs. BRAM Latency. Early prototypes used BRAM for the L1 scratchpad to save flip-flop resources. One-cycle read latency in BRAM introduced pipeline bubbles between `WRITE` and the subsequent `LOADB` that consumed the results. Moving to a purely flip-flop-based scratchpad eliminated the latency, at the cost of ≈ 3000 additional FFs—acceptable on the XC7Z045.

AV Synchronisation. Naive back-to-back memcpy of 2.76 MB video frames caused AXI interconnect stalls that starved the audio DMA FIFO, producing audible glitches. The 30 ms inter-frame gate in the PS firmware enforces a minimum spacing between heavy memory operations. Audio is explicitly made the master clock: the video frame index is derived from the audio chunk counter, not from a wall-clock timer, which absorbs the variable latency of DDR3 arbitration.

ADV7511 HPD Race. The ADV7511 requires the display to assert Hot Plug Detect (HPD) before accepting I2C configuration. The sequencer's first substantive transaction reads HPD state and polls with a 10 ms backoff until asserted, preventing the sequencer from issuing configuration writes to a display that is not ready. A NACK received at any subsequent transaction causes a full restart from transaction 0, covering the case where the display de-asserts HPD unexpectedly.

Fixed-Point Range and Zoom. Q8.24 provides 8 bits of integer range ($[-128, 127]$) and 24 bits of fractional precision ($2^{-24} \approx 60$ ppb). As the Mandelbrot zoom increases, Δx and Δy decrease toward zero. The zoom program terminates via a `BGE` guard when $\Delta y \leq 0$ to prevent underflow. Deeper zoom would require either wider fixed-point representation or migration to floating-point, neither of which is supported in the current architecture.

8. Conclusion

A 32-core SIMD GPGPU has been demonstrated on the PL fabric of a Xilinx Zynq-7000 ZC706, achieving 2.37 GOPS peak throughput at 74.25 MHz in Q8.24 fixed-point arithmetic. The design covers the full stack: a custom 16-opcode ISA and two-pass assembler, a parameterised MAC array with 3072-bit internal bus, a flat flip-flop scratchpad for zero-latency operand access, dual-port VRAM with hardware double-buffering, and a complete HDMI output pipeline including bitbanged I2C ADV7511 initialisation. On the PS side, bare-metal firmware provides FAT-32 SD card media loading, AXI VDMA-based video output, AXI DMA audio streaming, and audio-master AV synchronisation.

The Mandelbrot renderer demonstrates the GPU's programmability for iterative fixed-point algorithms, achieving approximately 0.7 FPS at 1280×720 with concurrent background video playback. The matrix multiplication example demonstrates the ISA's ability to express dot-product accumulation with explicit C-accumulation control. Both programs expose limitations of the current architecture—primarily the 16-bit immediate constraint, the absence of per-lane predication, and the fixed-point range ceiling—which motivate future extensions toward wider immediates, floating-point support, and a deeper instruction BRAM.

References

[1] L. Newhouse and H. Cui, "Minimalist GPU on an FPGA," MIT 6.111 Final Project Report, 2022. [Online]. Available: https://github.com/Arongil/FPGA-GPU

[2] Analog Devices, Inc., *ADV7511 Low-Power HDMI Transmitter Hardware User's Guide*, Rev. D, Jul. 2011. [Online]. Available: https://www.analog.com/media/en/technical-documentation/user-guides/ADV7511_Hardware_Users_Guide.pdf

[3] Analog Devices, Inc., *ADV7511 HDMI Transmitter Programming Guide*, Rev. 0. [Online]. Available: https://www.analog.com/media/en/technical-documentation/user-guides/ADV7511_Programming_Guide.pdf

[4] Analog Devices, Inc., *UG-235: EVAL-ADV7511 Evaluation Board User Guide*. [Online]. Available: https://www.analog.com/media/en/technical-documentation/user-guides/UG-235.pdf

[5] Xilinx, Inc., *UG954: ZC706 Evaluation Board for the Zynq-7000 XC7Z045 SoC User Guide*, v1.7, 2019. [Online]. Available: https://www.xilinx.com/support/documentation/boards_and_kits/zc706/ug954-zc706-eval-board-xc7z045-ap-soc.pdf

[6] Xilinx, Inc., *PG020: AXI Video Direct Memory Access v6.2 LogiCORE IP Product Guide*, Nov. 2016. [Online]. Available: https://www.xilinx.com/support/documentation/ip_documentation/axi_vdma/v6_2/pg020_axi_vdma.pdf

[7] Xilinx, Inc., *PG021: AXI DMA v7.1 LogiCORE IP Product Guide*. [Online]. Available: https://docs.amd.com/r/en-US/pg021_axi_dma

[8] Xilinx, Inc., *UG1037: Vivado Design Suite AXI Reference Guide*, v4.0, 2017. [Online]. Available: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf

[9] Xilinx, Inc., *UG585: Zynq-7000 SoC Technical Reference Manual*, v1.13, 2018. [Online]. Available: https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf

[10] Consumer Electronics Association, *CEA-861-F: A DTV Profile for Uncompressed High Speed Digital Interfaces*, 2013.

[11] C. Minami (ChaN), "FatFs – Generic FAT Filesystem Module," R0.15, 2022. [Online]. Available: http://elm-chan.org/fsw/ff/00index_e.html